C Programming

Saturday, 22 November, 2025 11:29 AM

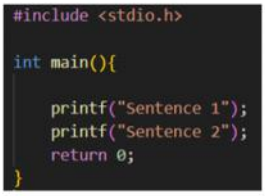
C files are stored using .c

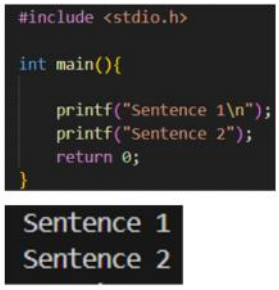
Main() function is the entry point of the function.

int main(){
}
Where the block of code is written inside {}

To print use printf()

The end character is in the same line (while in python its next line).
To make the next printf command to print on the next line use the new line character `\n`.
For printing a text, enter the text between "" quotes (raises error if it is in '')





Each sentence should end with ;

At the end of the main function,

return 0;
In C the main function is expected to return an integer. Zero indicates that the program ran successfully.
Non-zero value typically indicates an error.
In older versions of C, removing this statement will result in undefined behavior.

Comments,

Single line comments in C use // (2 slash)
Multi line comments in C use /* */ (Text in-between /* and */)

Variables,

A reusable container for a value. Behaves as if it were the value it contains.
To insert a Variable anywhere use `%` which is a format.

Data Type	Explanation	Code	Print	Comments
Int	Whole numbers (4 bytes)	int age = 25;	printf("You are %d years old", age);	d is for decimal (Whole numbers).
Float	Single precision decimal number (4 bytes)	float gpa = 3.7;	printf("Your gpa is %f", gpa);	f is for float (can only up to 6-7 decimal places).
Double	Double precision decimal number (8 bytes)	double pi = 3.141592653589793	printf("The value of pi is %lf", pi);	lf is for long float (can up to 15 - 16 decimal places). Can use f also.
Char (1 Character)	Single character (1 byte)	char grade = 'A'	printf("Your Grade is %c", grade);	c is for character (1 character). Single characters are entered in ' ' (Single quotes) or it raises error.
Char[] (More than 1 character)	Array of characters (size varies) (aka string)	char name[] = "Kiru"	printf("My Name is %s", name);	[] is an array (which can store more than 1 value). s is for String (more than 1 character). These are entered in " " (Double quotes) or it raises error.
Bool (Boolean)	true or false (1 byte) (requires <stdbool.h>)	bool isOnline = true;	Used in if statements.	For boolean we have to do `#include <stdbool.h>`.

Format Specifiers,

Special Tokens that begin with a % symbol, followed by a character that specifies the data type (d, f, lf, c, s) and optional modifiers (width, precision, flags). They control how data is displayed or interpreted.

Data type	Format Specifier
Int	%d (d is decimal).
Float	%f (f is float).
Double	%lf (lf is long float). For printing we can use %f also.
Char	%c (c is character). Used only for single characters.
Char[]	%s (s is string). Used for many characters.
Bool	%d (decimal is used as for printing it returns 1 or 0).

Width,

The number between the % and the character specifies the width

```
int num1 = 1;
    int num2 = 10;
    int num3 = 100;
    printf("%4d \n", num1);         // The number between the % and the character specifies the width
    printf("%4d \n", num2);
    printf("%4d \n", num3);
    printf("\n%-4d \n", num1);
      // The number between the % and the character specifies the width (the - sign left justifies)
    printf("%-4d \n", num2);
    printf("%-4d \n", num3);
    printf("\n%04d \n", num1);
      // The number between the % and the character specifies the width (preciding the number with 0 will make it print with
leading zeros)
    printf("%04d \n", num2);
    printf("%04d \n", num3);
```

Precision,

The numbers Specified after '.'

```
float price1 = 19.99;
    float price2 = -1.50;
    float price3 = 99.99;
    printf("%.2f\n", price1);
       // The numbers after the '.' specifies how many digits to display (inbetween % and the character)
    printf("%.2f\n", price2);
    printf("%.1f\n", price3);
       // The output will be rounded if we specify the decimal places lesser than the number stor
```

Flags,

Used to Flag a condition.

```
int num4 = 1;
    int num5 = 10;
    int num6 = -100;
    printf("%+d \n", num4);       // The + sign between the % and the character print '+' is the number is positive
    printf("%+d \n", num5);
    printf("%+d \n", num6);
```

Width + Precision + Flags,

```
float price4 = 19.99;
    float price5 = -1.50;
    float price6 = 99.99;
    printf("%+7.2f\n", price4);
    printf("%+7.2f\n", price5);
    printf("%+7.1f\n", price6);
    // Where '+' is the Flag, '7' is the Width, and '.2' & '.1' are Precision
```

Arithmetic Operations,

Symbol	Operation	Example

+	Addition	`int x = 2;`
		`int y = 3;`
		`int z = 0;`
		`z = x + y;`
		`printf("Addition = %d \n", z);`

-	Subtraction	`int x = 2;`
		`int y = 3;`
		`int z = 0;`
		`z = x - y;`
		`printf("Subtraction = %d\n", z);`

*	Multiplication	`int x = 2;`
		`int y = 3;`
		`int z = 0;`
		`z = x * y;`
		`printf("Multiplication = %d\n", z);`

/	Division	`z = x / y;`
		`printf("Division = %d \n", z);  // Division of integers cannot store decimal places so it gives 0`
		`float a = 2;`
		`float b = 3;`
		`float c = 0;`
		`c = a / b;`
		`printf("Division = %f \n", c);`
		`c = x / b;  // x is an integer`
		`printf("Division = %f \n", c);`
		`// Either 1 value that you are dividing has to be float or else it will retain 0`

%	Modulus	`int d = 10;`
		`int e = 3;`
		`int f = 0;`
		`f = d % e;`
		`printf("Modulus = %d \n", f);`

++	Increment (Increases the value by 1)	`d++;  // Increments by 1`
		`printf("Increment %d \n", d);`

--	Decrement (Decreases the value by 1)	`e--;  // Decrements by 1`
		`printf("Decrement %d \n", e);`

Augmented Assignment Operators,

Symbol	Operation	Example

+=	Adds the value to the variable	`x+=2;  // Reassigning With Addition`
		`printf("Reassigning x(2) value + 3 = %d \n", x);`

-=	Subtracts the value from the variable	`x-=1;  // Reassigning With Subtraction`
		`printf("Reassigning x(4) value - 1 = %d \n", x);`

*=	Multiples the value to the variable	`x*=3;  // Reassigning With Multiplication`
		`printf("Reassigning x(3) value * 3 = %d \n", x);`

/=	Divides the value from the variable	`x/=3;  // Reassigning With Division`
		`printf("Reassigning x(4) value / 3 = %d \n", x);`

User Input,

To get user input use scanf(); function.
The printf() used before scanf() to tell the user what to enter as in scanf() you can't use to prompt any message.
The '&' used in scanf() function infront of the variable name is the 'address of operator' (it's a format to call the variable name in the function).
For many characters, since scanf() cant take whitespaces for input, we have to use a different function called fgets() which means file get string.

You can initialize empty variables, but if you accidently print variables which doesn't hold any data, it results in undefined behavior.
Best practice is to assign values (To not get undefined behavior even if incase we accidently print it before assigning a value).

Data Type	Code	Explanation

Int	`int age = 0;`
	`printf("Enter You Age : ");`
	`scanf("%d", &age);`
	`printf("Your Age is : %d \n", age);`
		Used for integers.
		The 'd' is for decimal (cannot be used for float).

Float	`float gpa = 0.0f;`
	`printf("Enter Your GPA : ");`
	`scanf("%f", &gpa);`
	`printf("Your GPA is : %.1f \n", gpa);`
		The 'f' is used to indicate it is a float (not used for double).
		Float value contains up to 6 - 7 digits.

Double	`double pi = 0.0;`
	`printf("Enter Pi Value : ");`
	`scanf("%lf", &pi);`
	`printf("Pi value is : %lf \n", pi);`
		Double is used when it's a long floating point numbers.
		The 'lf' means it is a long float (15 - 16 digits).

Char (Single Character)	`char grade = '\0';`
	`printf("Enter Your Grade : ");`
	`scanf(" %c", &grade);`
	`printf("Your Grade is : %c \n", grade);`
		'\0' is a Null Terminator.
		The newline character (\n) within the input buffer gets directly assigned into grade.
		To avoid this add a ' ' infront of %c to tell the program to skip 1 space

Char[] (Many Characters)	`char name[30] = "";`
	`getchar();`
	`printf("Enter Your Name : ");`
	`//scanf("%s", &name);`
	`fgets(name, sizeof(name), stdin);`
	`name[strlen(name) - 1] = '\0';`
	`printf("Your Name is : %s \n", name);`
		Have to assign a size if not assigning value directly.
		getchar() function Clears out newline character (used for fgets).
		Scanf can't read any whitespaces, So if we enter the full name it only saves the first name.
		The `` `name[strlen(name) - 1] = '\0';` `` Used to remove the newline character input taken in fgets function.
		fgets - file get string, stdin - Standard Input.
		In fgets() function, first enter the variable name, then the size (we can put the size directly but if we change it, we have to update everywhere manually so use the sizeof() function), and then stdin which means standard input.

Sample Program 1: Shopping Cart Program,

A simple program that requests input from user like Item name, price of item and quantity and produces the total price to pay.

Code:

```c
int main(){
    char item[50]="";
    float price = 0.0f;
    int quantity = 0;
    char currency = '$';
    float total = 0.0f;
    printf("What item would you like to buy? ");
    fgets(item, sizeof(item), stdin);
    item[strlen(item) - 1]='\0';
    printf("What is the price for each item? ");
    scanf("%f", &price);
    printf("How many would you like to buy? ");
    scanf("%d", &quantity);
    total = price * quantity;
    printf("You bought %d %s \n", quantity, item);
    printf("Total Price : %c %.2f",currency, total);
    return 0;
}
```

Output:

```
What item would you like to buy? Pizza
What is the price for each item? 10.01
How many would you like to buy? 3
You bought 3 Pizza
Total Price : $ 30.03
```

Sample Program 2 : Mad libs game

Code:

```c
int main(){
    char noun[50] = "";     //Person place or thing
    char verb[50] = "";     // action
    char adj1[50] = "";     // describes something
    char adj2[50] = "";
    char adj3[50] = "";
```

```c
int main(){
    char noun[50] = "";     //Person place or thing
    char verb[50] = "";     // action
    char adj1[50] = "";     // describes something
    char adj2[50] = "";
    char adj3[50] = "";

    printf("Enter an Adjective (Description) : ");
    fgets(adj1, sizeof(adj1), stdin);
    adj1[strlen(adj1) - 1] = '\0';
    printf("Enter a Noun (animal or person) : ");
    fgets(noun, sizeof(noun), stdin);
    noun[strlen(noun) - 1] = '\0';
    printf("Enter an Adjective (Description) : ");
    fgets(adj2, sizeof(adj2), stdin);
    adj2[strlen(adj2) - 1] = '\0';
    printf("Enter a Verb (Action, ending with -ing) : ");
    fgets(verb, sizeof(verb), stdin);
    verb[strlen(verb) - 1] = '\0';
    printf("Enter an Adjective (Description) : ");
    fgets(adj3, sizeof(adj3), stdin);
    adj3[strlen(adj3) - 1] = '\0';

    printf("\n Today I went to a %s zoo\n", adj1);
    printf("In an exhibit, I saw a %s \n", noun);
    printf("%s was %s and %s \n", noun, adj2, verb);
    printf("I was %s \n", adj3);

    return 0;
}
```

Output:

```
Enter an Adjective (Description) : Slow
Enter a Noun (animal or person) : Sonic
Enter an Adjective (Description) : Blue
Enter a Verb (Action, ending with -ing) : Sleeping
Enter an Adjective (Description) : Bored

 Today I went to a Slow zoo
In an exhibit, I saw a Sonic
Sonic was Blue and Sleeping
I was Bored
```

Math Functions,

Need to include <math.h> header file, or else it will raise an error.

```c
int main(){
    int x = 9;
    int y = 0;
    y = sqrt(x);                        // Square Root
    printf("Square Root : %d \n", y);
    y = pow(x, 2);                      // Power
    printf("Power : %d \n", y);
    float a = 3.14;
    float b = 3.14;
    float c = 3.99;
    a = round(a);                       // Rounds up to nearest number
    printf("Round : %f \n", a);
    b = ceil(b);                        // Rounds up to the higher number
    printf("Ceil : %f \n", b);
    c = floor(c);                       // Rounds down to the lower number
    printf("Floor : %f \n", c);
    int z = -3;
    z = abs(z);                         // Absolute of a value (gives the distance from 0 in + value)
    printf("Absolute : %d \n", z);
    float d = 3.0;
    d = log(3);                         // Natural log of the number
    printf("Natural Log : %f \n", d);
    int r = 45;     // Taken in Rad
    float t = 0.0f;
    t = sin(r);
    printf("Sin : %f \n", t);
    t = cos(r);
    printf("Cos : %f \n", t);
    t = tan(r);
```

```
    printf("Sin : %f \n", t);
    t = cos(r);
    printf("Cos : %f \n", t);
    t = tan(r);
    printf("Tan : %f \n", t);

    return 0;
}
```

Sample Program 3 : Circle Calculator Program

A simple program which takes the radius as the input and produces the Area, Surface Area, and the Volume.

Code:

```
int main(){
    double radius = 0.0;
    double area = 0.0;
    double sarea = 0.0;
    double volume = 0.0;

    printf("Enter Radius of the circle : ");
    scanf("%lf", &radius);

    const double pi = 3.1415926535;
    area = pi * pow(radius, 2);
    printf("Area of the Circle : %.2lf \n", area);

    sarea = 4 * pi * pow(radius, 2);
    printf("Surface Area of the Circle : %.2lf \n", sarea);

    double r3 = 0.0;
    volume = (4/3) * pi * pow(radius, 3);
    printf("Volume of Sphere : %.2lf \n", volume);

    return 0;
}
```

Output:

```
Enter Radius of the circle : 10
Area of the Circle : 314.16
Surface Area of the Circle : 1256.64
Volume of Sphere : 3141.59
```

Sample Program 4 : Compound Interest Calculator

Code :

```
int main(){
    float principle = 0.0f;
    float rate = 0.0f;
    float time = 0.0f;
    float compound = 0.0f;
    float amount = 0.0f;

    printf("Enter Principle Amount (P) : ");
    scanf("%f", &principle);
    printf("Enter Interest Rate  (r) : ");
    scanf("%f", &rate);
    rate = rate/100;
    printf("Enter number of years (t) : ");
    scanf("%f", &time);
    printf("Enter number of times compounded per year (n) : ");
    scanf("%f", &compound);

    float a1 = 0.0f;
    float a2 = 0.0f;
    a1 = 1 + (rate/compound);
    a2 = compound*time;
    amount = principle * pow(a1, a2);
    printf("After %.2f years, the total will be %.2f", time, amount);

    return 0;
}
```

```
    printf("After %.2f years, the total will be %.2f", time, amount);

    return 0;
}
```

Output :

```
Enter Principle Amount (P) : 1000
Enter Interest Rate  (r) : 10
Enter number of years (t) : 2
Enter number of times compounded per year (n) : 1
After 2.00 years, the total will be 1210.00
```

If Statements,

Does the code if the condition is true.

Example code 1,

```c
int main(){
    int age = 0;
    printf("Enter Age : ");
    scanf("%d", &age);

    if(age>=65){
        printf("You are a Senior.");
    }
    else if(age<0){
        printf("Age cannot be Negative.");
    }
    else if(age == 0){
        printf("You are a new born");
    }
    else if(age>=18){
        printf("You are an Adult.");
    }
    else{
        printf("You are a child.");
    }

    return 0;
}
```

Example code 2,

```c
int main(){
    bool isStudent = true;

    if(isStudent==true){
        printf("You are a student.");
    }
    else{
        printf("You are not a student.");
    }

    return 0;
}
```

Example code 3,

```c
int main(){
    char name[50] = "";

    printf("Enter your name : ");
    fgets(name, sizeof(name), stdin);
    name[strlen(name) - 1] = '\0';

    if(strlen(name)==0){
        printf("Name cannot be empty.");
    }
    else{
        printf("Hello %s!", name);
    }

    return 0;
}
```

Sample Program 5 : Weight Conversion Program

A simple code that converts kilograms to pounds and pounds to kilograms

Code :

```c
int main(){
    int s = 0;
    float kg = 0.0f;
    float p = 0.0f;
    printf("1. Kilograms to Pounds \n2. Pounds to Kilograms
\n")printf("Enter your selection (1 or 2) : ");
    scanf("%d", &s);

    if(s == 1){
        printf("Enter Kilograms (Kg) : ");
        scanf("%f", &kg);

        p = kg * 2.2;
        printf("Pounds (lb) : %.2f", p);
    }
    else if(s == 2){
        printf("Enter Pounds (lb) : ");
        scanf("%f", &p);

        kg = p/2.2;
        printf("Kilograms (Kg) : %.2f", kg);
    }
    else{
        printf("Enter a valid choice");
    }

    return 0;
}
```

```
        printf("Enter a valid choice");
    }

    return 0;
}
```

Output :

```
1. Kilograms to Pounds
2. Pounds to Kilograms
Enter your selection (1 or 2) : 1
Enter Kilograms (Kg) : 10
Pounds (lb) : 22.00
```

```
1. Kilograms to Pounds
2. Pounds to Kilograms
Enter your selection (1 or 2) : 2
Enter Pounds (lb) : 22
Kilograms (Kg) : 10.00
```

Sample Program 6 : Temperature Conversion Program

A simple program that converts Celsius to Fahrenheit and Fahrenheit to Celsius

Code :

```
int main(){
    int choice = 0;
    float c = 0.0f;
    float f = 0.0f;
    printf("1. Celsius to Fahrenheit \n2. Fahrenheit to Celsius
\n")printf("Enter Choice (1 or 2) : ");
    scanf("%d", &choice);
    if(choice==1){
        printf("Enter Celsius (C) : ");
        scanf("%f", &c);
        f = ((9.0/5.0)*c)+32;
        printf("Fahrenheit (F) : %.2f", f);
    }
    else if(choice == 2){
        printf("Enter Fahrenheit (F) : ");
        scanf("%f", &f);
        c = ((f - 32)*5.0)/9.0;
        printf("Celsius (C) : %.2f", c);
    }
    else{
        printf("Enter a Valid Option");
    }

    return 0;
}
```

Output :

```
1. Celsius to Fahrenheit
2. Fahrenheit to Celsius
Enter Choice (1 or 2) : 1
Enter Celsius (C) : 25
Fahrenheit (F) : 77.00
```

```
1. Celsius to Fahrenheit
2. Fahrenheit to Celsius
Enter Choice (1 or 2) : 2
Enter Fahrenheit (F) : 77
Celsius (C) : 25.00
```

Switch,

An alternative to using many if and elseif statements.

More efficient when working with fixed integer values or characters (single characters).

`break` in each case is important as it ends the switch if the condition is met, or else it proceeds till switch end.

The `default` case is run if there is no matching cases found.

Example 1 (with int),

```
int main(){
    int dayOfWeek=0;
    printf("Enter Day of the week (1-7) : ");
    scanf("%d", &dayOfWeek);

    switch(dayOfWeek){
        case 1:
            printf("It is Monday");
            break;
        case 2:
            printf("It is Tuesday");
            break;
        case 3:
            printf("It is Wednesday");
            break;
        case 4:
```

Example 2 (with char),

```
int main(){
    char dayOfWeek='\0';
    printf("Enter Day of the week (M, T, W, H, F, S, U) : ");
    scanf("%C", &dayOfWeek);

    switch(dayOfWeek){
        case 'M':
            printf("It is Monday");
            break;
        case 'T':
            printf("It is Tuesday");
            break;
        case 'W':
            printf("It is Wednesday");
            break;
        case 'H':
```

```
        case 3:
            printf("It is Wednesday");
            break;
        case 4:
            printf("It is Thursday");
            break;
        case 5:
            printf("It is Friday");
            break;
        case 6:
            printf("It is Saturday");
            break;
        case 7:
            printf("It is Sunday");
            break;
        default:
            printf("Enter a number (1-7)");
    }

    return 0;
}
```

```
        case 'W':
            printf("It is Wednesday");
            break;
        case 'H':
            printf("It is Thursday");
            break;
        case 'F':
            printf("It is Friday");
            break;
        case 'S':
            printf("It is Saturday");
            break;
        case 'U':
            printf("It is Sunday");
            break;
        default:
            printf("Enter a Valid Character (M, T, W, H, F, S, U)");
    }

    return 0;
}
```

Nested If statements,

Example,

```
int main(){
    float price = 10.00;
    bool isStudent = false;
    bool isSenior = false;
    char stu = '\0';
    char se = '\0';

    printf("Are you a Student? (Y/N) ");
    scanf("%c", &stu);
    printf("Are you a Senior? (Y/N) ");
    scanf(" %c", &se);

    if(stu == 'Y'){
        isStudent = true;
    }
    if(se == 'Y'){
        isSenior = true;
    }

    if(isStudent == true){
        if(isSenior == true){
            printf("You have a Senior discount of 20%% \n");
            printf("You have a total discount of 30%% \n");
            price = price * 0.7;
        }
        else{
            printf("You have a Student discount of 10%%
\n");         price = price * 0.9;
        }
    }
    else if(isStudent == false){
        if(isSenior == true){
            printf("You have a Senior discount of 20%% \n");
            price = price * 0.8;
        }
    }
    printf("The price of the ticket is : $ %.2f", price);

    return 0;
}
```

Sample Program 7 : Calculator Program

Code :

```
int main(){
    double num1 = 0.0;
    double num2 = 0.0;
    double result = 0.0;
    char operator = '\0';
    bool valid = false;
```

```c
int main(){
    double num1 = 0.0;
    double num2 = 0.0;
    double result = 0.0;
    char operator = '\0';
    bool valid = false;

    printf("Enter Number 1 : ");
    scanf("%lf", &num1);
    printf("Enter Number 2 : ");
    scanf("%lf", &num2);
    printf("Enter Operator (+, -, /, *) : ");
    scanf(" %c", &operator);

    switch(operator){
        case '+':
            result = num1 + num2;
            valid = true;
            break;
        case '-':
            result = num1 - num2;
            valid = true;
            break;
        case '/':
            if(num2==0.0){
                printf("Division by Zero is not valid");
            }
            else{
                result = num1 / num2;
                valid = true;
            }
            break;
        case '*':
            result = num1 * num2;
            valid = true;
            break;
        default:
            printf("Please Enter a Valid Operator (+, -, /, *).");
    }
    if(valid == true){
        printf("Result = %lf", result);
    }

    return 0;
}
```

Output:

```
Enter Number 1 : 10
Enter Number 2 : 20
Enter Operator (+, -, /, *) : +
Result = 30.000000
```

```
Enter Number 1 : -10
Enter Number 2 : 20
Enter Operator (+, -, /, *) : -
Result = -30.000000
```

```
Enter Number 1 : 20
Enter Number 2 : 10
Enter Operator (+, -, /, *) : -
Result = 10.000000
```

```
Enter Number 1 : 10
Enter Number 2 : 20
Enter Operator (+, -, /, *) : -
Result = -10.000000
```

```
Enter Number 1 : 10
Enter Number 2 : 5
Enter Operator (+, -, /, *) : /
Result = 2.000000
```

```
Enter Number 1 : 10
Enter Number 2 : 0
Enter Operator (+, -, /, *) : /
Division by Zero is not valid
```

```
Enter Number 1 : 10
Enter Number 2 : 2
Enter Operator (+, -, /, *) : *
Result = 20.000000
```

```
Enter Number 1 : 10
Enter Number 2 : 10
Enter Operator (+, -, /, *) : x
Please Enter a Valid Operator (+, -, /, *).
```

Logical Operators,

Used to compare or modify Boolean expressions.

Expression	Symbol	Example	Explanation
AND	&&	`if(temp>0 && temp<30)`	Used when you want to check if it is within a range. Both the conditions has to be true.
OR	||	`if(temp<=0 || temp >=30)`	Used when you want to check if it is outside a range. Either 1 of the conditions has to be true.
NOT	!	`if(!isSunny)`	Reverses any Boolean expression. true => false ; false => true

Example 1 - AND (&&),

Example 2 - OR (||),

Example 3 - NOT (!),

Example 1 - AND (&&),

```c
int main(){
    int temp = 50;

    if(temp>0 && temp<30){
        printf("The temperature is good");
    }
    else{
        printf("The temperature is bad");
    }

    return 0;
}
```

Example 2 - OR (||),

```c
int main(){
    int temp = 20;

    if(temp<=0 || temp >=30){
        printf("The temperature is bad");
    }
    else{
        printf("The temperature is good");
    }

    return 0;
}
```

Example 3 - NOT (!),

```c
int main(){
    bool isSunny = false;

    if(!isSunny){
        printf("It is sunny.");
    }
    else{
        printf("It is cloudy");
    }

    return 0;
}
```

Functions (pass by value),

A reusable section of code that can be called again (invoked).
Arguments can be sent into a function so that it can use them.
A function is defined outside the main() function used 'void'.
To pass arguments into a function, you describe the parameter's data type also.

Argument - What you send into the function while calling it.
Parameters - What the function expects to receive.

Example,

```c
void happybirthday(char name[], int age){
    printf("\n Happy Birthday");
    printf("\n Happy Birthday");
    printf("\n Happy Birthday to %s", name);
    printf("\n Happy Birthday");
    printf("\n You are now %d years old.", age);
}
int main(){
    char name[50] = "";
    int age = 0;

    printf("Enter Person's Name : ");
    fgets(name, sizeof(name), stdin);
    name[strlen(name) - 1] = '\0';
    printf("Enter Person's Age : ");
    scanf("%d", &age);

    happybirthday(name, age);

    return 0;
}
```

If you call the function without any arguments, it will raise an error (only if you have parameters set up).
If you call the function with incorrect order of the arguments, it raises an error.

Return Keyword,

Returns a value back when you call a function.
To use return, we cannot use 'void',

Use 'int' if it is a integer.
Use 'double' if it is double.
Use 'bool' if it is Boolean.
Use 'void' if it is none.

Example 1 - Int,

```c
int square(int num){
    return num * num;
}
int main(){
    int x = square(2);
    int y = square(3);
    int z = square(4);
```

Example 2 - Double,

```c
double square(double num){
    return num * num;
}
int main(){
    double x = square(2.12
3); double y = square(3.12
```

Example 3 - bool,

```c
bool agecheck(int age){
    if(age>=18){
        return true;
    }
    else{
        return false;
    }
}
int main(){
    int age = 0;
```

```c
int main(){
    int x = square(2);
    int y = square(3);
    int z = square(4);

    printf("%d \n",
x); printf("%d \n",
y); printf("%d \n",
z);
    return 0;
}
```

```c
int main(){
    double x = square(2.12
3); double y = square(3.12
3); double z = square(4.12
3);
    printf("%lf \n", x);
    printf("%lf \n", y);
    printf("%lf \n", z);

    return 0;
}
```

```c
    }
}
int main(){
    int age = 0;

    printf("Enter Your Age : ");
    scanf("%d", &age);

    if(agecheck(age)){
        printf("You are an Adult");
    }
    else{
        printf("You are a Child");
    }

    return 0;
}
```

Variable Scope,

Refers to where a variable is recognized and accessible.

Variables can share the same name if they are in different scopes { }.

Example 1,

```c
int main(){
    int result = 0;
    int result = 1;

    return 0;
}
```

This will raise an error as there are 2 variables sharing the same name in the same scope.

Example 2,

```c
int add(int x, int y){
    int result = x + y;
    return result;
}
int main(){
    int result = add(3, 4);
    printf("Result = %d", result);

    return 0;
}
```

This will work without raising any error.
(functions cannot see inside another function)

Local Scope,

Variables defined locally within functions (other parts of the program cannot access them).

Global scope,

Variables defined outside a function (All parts of the program has access to it).

Recommended to avoid since it is hard to debug (cause another function can accidently modify its value).

```c
int result = 0; // Global Scope
int add(int x, int y){
    result = x + y;
    return result;
}
int main(){
    result = add(3, 4);
    printf("Result = %d", result);

    return 0;
}
```

Function Prototypes,

A statement that is listed before the main function.

It provides information about a function's name, its return type, and its parameters.

Advantages,

Enables Type Checking

Allows functions to be used before they are defined.

Improves readability, organization, and helps prevent errors.

In C, the program is read from top to bottom, so if a function is called before it is defined it raises and error.

```c
void hello(char name[], int age);       // Function Prototype
bool agecheck(int age);                 // Function Prototype
int main(){                             // Main Function
    char name[50] = "";
    int age = 0;
    printf("Enter Name : ");
    fgets(name, sizeof(name), stdin);
    name[strlen(name) - 1] = '\0';
    printf("Enter Age : ");
    scanf("%d", &age);
    hello(name, age);
    if(agecheck(age)){
        printf("You are Old enough to work are Krusty Krab");
    }
    else{
        printf("You need to be 16+ to work at Krusty Krab");
    }

    return 0;
}
void hello(char name[], int age){       // Function
    printf("Hello %s \n", name);
    printf("You are %d years old \n", age);
}
bool agecheck(int age){                 // Function
    return age >= 16;
}
```

While Loop,

Repeats a set of code WHILE the condition is true (Number of loops is not known).
Useful for taking in valid user inputs (Example 2) (Used for checking input)
Example 1 (Can be converted into a For loop) ,

```c
int main(){
    int num = 0;
    printf("Enter How many times you want to print : ");
    scanf("%d", &num);
    int i = 1;
    while(i<=num){
        printf("Hello, Count : %d \n", i);
        i += 1;
    }

    return 0;
}
```

Prints "Hello" the number of times the user enters

Example 2,

```c
int main(){
    int num = 0;
    while(num <=0){
        printf("Enter a number greater than zero : ");
        scanf("%d", &num);
    }

    return 0;
}
```

Goes into the loop until the user enters a number greater than zero. (Useful for taking in Valid User Input).

Example 3,

```c
int main(){
    char name[50] = "";
    printf("Enter Your Name : ");
    fgets(name, sizeof(name), stdin);
    name[strlen(name) - 1] = '\0';

    while(strlen(name)==0){
        printf("Name Cannot be empty. \n");
        printf("Enter Your Name : ");
        fgets(name, sizeof(name), stdin);
        name[strlen(name) - 1] = '\0';
    }
    printf("Hello %s", name);

    return 0;
```

Example 4,

```c
int main(){
    bool isRunning = true;
    char response = '\0';

    while(isRunning){
        printf("You are playing a game. \n");
        printf("Would you like to continue? (y = Yes, n = No) : ");
        scanf(" %c", &response);
        if(response != 'Y' && response != 'y'){
            isRunning = false;
        }
    }
    printf("You Exit the game.");
```

```
    }
    printf("Hello %s", name);

    return 0;
}
```

Goes into the loop until the user enters a valid name.
(Useful for taking in Valid User Input).

```
    }
    printf("You Exit the game.");

    return 0;
}
```

Uses Bool to exit the loop (Condition should be true to enter the loop).

Do-While loop,

Does the loop once and then checks the condition.

```
int main(){
    int num = 1;    // Number set higher that 0
    do{
        printf("Enter a number greater than zero : ");
        scanf("%d", &num);
    }
    while(num <=0);

    return 0;
}
```

For Loop,

Repeats a set of c ode for a limited number of times (Number of loops is known).
for(Initialization, Condition, Update)

Example 1,

```
int main(){
    int num = 0;
    printf("Enter Count : ");
    scanf("%d", &num);
    for(int i = 1; i<=num; i+=1){
        printf("Count : %d \n", i);
    }

    return 0;
}
```

```
Enter Count : 10
Count : 1
Count : 2
Count : 3
Count : 4
Count : 5
Count : 6
Count : 7
Count : 8
Count : 9
Count : 10
```

Example 2,

```
int main(){
    int num = 0;
    printf("Enter Count : ");
    scanf("%d", &num);
    printf("Countint in reverse \n");   // Instead of using 'i' we can use 'num' directly
    for(int i = num; i>=0; i-=1){       // for(num; num>=0; num-=1){
        printf("Count : %d \n", i);     // printf("Count : %d \n", num);

    }

    return 0;
}
```

```
Enter Count : 10
Countint in reverse
Count : 10
Count : 9
Count : 8
Count : 7
Count : 6
Count : 5
Count : 4
Count : 3
Count : 2
Count : 1
Count : 0
```

Break & Continue,

Break - Breaks out of the loop (Stop)
Continue - Skip current cycle of loop (Skip)

```
int main(){
    for(int i=1; i<=10;i++){
        if(i==4){
            continue;       // Skips number 4
        }
        if(i==8){
            break;          // Breaks the loop if i = 8
        }
        printf("%d \n", i);
```

```c
int main(){
    for(int i=1; i<=10;i++){
        if(i==4){
            continue;       // Skips number 4
        }
        if(i==8){
            break;          // Breaks the loop if i = 8
        }
        printf("%d \n", i);
    }

    return 0;
}
```

```
1
2
3
5
6
7
```

Nested Loops,

Contains a Loop within a loop

Example 1,

```c
int main(){
    for(int i = 1; i <=3; i++){
        for(int j = 1; j < 10; j++){
        printf("%d ", j);
        }
        printf("\n");
    }

    return 0;
}
```

This Code prints 1 to 9 3 times

```
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
```

Example 2 (Multiplication table),

```c
int main(){
    for(int i = 1; i <= 10; i++){
        for(int j = 1; j <= 10; j++){
            printf("= ", i*j);
        }
        printf("\n");
    }

    return 0;
}
```

```
1   2   3   4   5   6   7   8   9  10
2   4   6   8  10  12  14  16  18  20
3   6   9  12  15  18  21  24  27  30
4   8  12  16  20  24  28  32  36  40
5  10  15  20  25  30  35  40  45  50
6  12  18  24  30  36  42  48  54  60
7  14  21  28  35  42  49  56  63  70
8  16  24  32  40  48  56  64  72  80
9  18  27  36  45  54  63  72  81  90
10 20  30  40  50  60  70  80  90 100
```

Example 3,

```c
int main(){
    int row = 0;
    int col = 0;
    char symbol = '\0';
    printf("Enter the Number of Rows : ");
    scanf("%d", &row);
    printf("Enter the Number of Cols : ");
    scanf("%d", &col);
    printf("Enter the Symbol : ");
    scanf(" %c", &symbol);
    for(int i = 1; i <= row; i++){
        for(int j = 1; j <= col; j++){
            printf("%c", symbol);
        }
        printf("\n");
    }
```

```
Enter the Number of Rows : 3
Enter the Number of Cols : 5
Enter the Symbol : !
!!!!!
!!!!!
!!!!!
```

```c
int main(){
    int row = 0;
    int col = 0;
    char symbol = '\0';
    printf("Enter the Number of Rows : ");
    scanf("%d", &row);
    printf("Enter the Number of Cols : ");
    scanf("%d", &col);
    printf("Enter the Symbol : ");
    scanf(" %c", &symbol);
    for(int i = 1; i <= row; i++){
        for(int j = 1; j <= col; j++){
            printf("%c", symbol);
        }
        printf("\n");
    }

    return 0;
}
```

```
Enter the Number of Rows : 3
Enter the Number of Cols : 5
Enter the Symbol : !
!!!!!
!!!!!
!!!!!
```

```
10  20  30  40  50  60  70  80  90 100
```

Random Numbers (Pseudo-Random),

Appear random but are determined by a mathematical formula that uses a seed value to generate a predictable sequence of numbers.
Advanced : Mersenne Twister or /dev/random .
While generating a pseudo-random number we are plugging in a base seed value into a mathematical formula to generate a sequence of numbers that appear random.
Base seed value in most cases - 0 or 1.
When we run the code ` printf("%d", rand()); ` This will give the same number '41' every time we run the code.
To create a seed value based on the current time,

```c
srand(time(NULL));
printf("%d", rand());
```

Use `RAND_MAX` to find the max number in the random function (changes according to the os and the complier used).
To get pseudo-random numbers

```c
srand(time(NULL));
int randomNum = rand() % 2; // Gives either 0 or 1
printf("%d", randomNum);
```

To get pseudo-random numbers using off-set (Adding a off-set of 1 cause 0 is a possible value)

```c
srand(time(NULL));
int randomNum = (rand() % 2) + 1; // Gives either 1 or 2
printf("%d", randomNum);
```

For Small Ranges,

```c
srand(time(NULL));
int min = 1;
int max = 6;
int randomNum = (rand() % max) + min;
printf("%d", randomNum);
```

For Greater Ranges,

```c
srand(time(NULL));
int min = 50;
int max = 100;
int randomNum = (rand() % (max - min + 1)) + min;   // Setting min as 50 and max as 100
printf("%d", randomNum);
```

Sample Program 8 : Number Guessing Game,

Program 1 - range 1 to 10 and gives 3 tries
Code :

```c
int main(){
    srand(time(NULL));
    int min = 1;
    int max = 10;
```

```c
int main(){
    srand(time(NULL));
    int min = 1;
    int max = 10;
    int randomNum = (rand() % (max - min + 1)) + min;
    int guessNum = 0;
    bool valid = false;

    printf("Number Guessing Game \n");
    for(int i = 1; i <= 3; i++){
        printf("Guess a Number between 1 to 10 : ");
        scanf("%d", &guessNum);
        if(guessNum == randomNum){
            printf("You have Guessed the Correct Number!");
            valid = true;
            break;
        }
        else{
            printf("Wrong Number, Guess again\n");
        }
    }
    while(valid==false){
        printf("You guessed the Wrong Number\n");
        printf("The Correct Number is %d", randomNum);
        break;
    }

    return 0;
}
```

Output :

```
Number Guessing Game
Guess a Number between 1 to 10 : 4
Wrong Number, Guess again
Guess a Number between 1 to 10 : 5
Wrong Number, Guess again
Guess a Number between 1 to 10 : 1
Wrong Number, Guess again
You guessed the Wrong Number
The Correct Number is 7
```

```
Number Guessing Game
Guess a Number between 1 to 10 : 6
Wrong Number, Guess again
Guess a Number between 1 to 10 : 3
You have Guessed the Correct Number!
```

Program 2 - range 1 to 100 and goes on till you get it

Code:

```c
int main(){
    srand(time(NULL));
    int min = 1;
    int max = 100;
    int randomNum = (rand() % (max - min + 1)) + min;
    int guessNum = 0;
    int tries = 1;
    int range1 = randomNum - 10;
    int range2 = randomNum + 10;
    bool valid = false;

    printf("Number Guessing Game \n");
    while(valid == false){
        printf("Guess a Number between 1 to 100 : ");
        scanf("%d", &guessNum);
        if(guessNum != randomNum){
            printf("You Guessed Wrong, Try Again. \n");
            if(range1<=guessNum && range2>=guessNum){
                printf("Close \n");
            }
            else if(guessNum<range1){
                printf("Too Low \n");
```

```
            }
            else if(guessNum<range1){
                printf("Too Low \n");
            }
            else if(guessNum>range2){
                printf("Too High \n");
            }
            tries += 1;
        }
        else{
            printf("You have Guessed the Correct Number! \n");
            printf("You did it in %d tries.", tries);
            valid = true;
            break;
        }
    }

    return 0;
}
```

Output:

```
Number Guessing Game
Guess a Number between 1 to 100 : 10
You Guessed Wrong, Try Again.
Too Low
Guess a Number between 1 to 100 : 100
You Guessed Wrong, Try Again.
Too High
Guess a Number between 1 to 100 : 50
You Guessed Wrong, Try Again.
Too Low
Guess a Number between 1 to 100 : 60
You Guessed Wrong, Try Again.
Too Low
Guess a Number between 1 to 100 : 70
You Guessed Wrong, Try Again.
Close
Guess a Number between 1 to 100 : 80
You Guessed Wrong, Try Again.
Close
Guess a Number between 1 to 100 : 90
You Guessed Wrong, Try Again.
Too High
Guess a Number between 1 to 100 : 71
You Guessed Wrong, Try Again.
Close
Guess a Number between 1 to 100 : 72
You Guessed Wrong, Try Again.
Close
Guess a Number between 1 to 100 : 75
You have Guessed the Correct Number!
You did it in 10 tries.
```

Sample Program 9 : Rock Paper Scissors Game

Code:

```
int main(){
    srand(time(NULL));
    int min = 1;
    int max = 3;
    int computer = (rand() % (max - min + 1)) + min;
    int human = 0;

    printf("ROCK PAPER SCISSORS GAME \n");
    printf("Choose an Option : \n1. Rock \n2. Paper \n3. Sissors \n");
    printf("Enter your Choice : ");
    scanf("%d", &human);
    switch(human){
        case 1:
            printf("You Choose Rock \n");
            break;
        case 2:
            printf("You Choose Paper \n");
```

```c
int main(){
    srand(time(NULL));
    int min = 1;
    int max = 3;
    int computer = (rand() % (max - min + 1)) + min;
    int human = 0;

    printf("ROCK PAPER SCISSORS GAME \n");
    printf("Choose an Option : \n1. Rock \n2. Paper \n3. Sissors \n");
    printf("Enter your Choice : ");
    scanf("%d", &human);
    switch(human){
        case 1:
            printf("You Choose Rock \n");
            break;
        case 2:
            printf("You Choose Paper \n");
            break;
        case 3:
            printf("You Choose Scissors \n");
            break;
        default:
            printf("Please enter a valid choice. \n");
            break;
    }
    switch(computer){
        case 1:
            printf("Computer Choose Rock \n");
            break;
        case 2:
            printf("Computer Choose Paper \n");
            break;
        case 3:
            printf("Computer Choose Scissors \n");
            break;
    }
    if(human == computer){
        printf("Its a Tie! \n");
    }
    else if((human == 1 && computer == 3) || (human == 3 && computer == 2) || (human == 2 && computer == 1)){
        printf("You Win! \n");
    }
    else if((human == 1 && computer == 2) || (human == 3 && computer == 1) || (human == 2 && computer == 3)){
        printf("You Lose! \n");
    }
    return 0;
}
```

Output:

```
ROCK PAPER SCISSORS GAME
Choose an Option :
1. Rock
2. Paper
3. Sissors
Enter your Choice : 1
You Choose Rock
Computer Choose Scissors
You Win!
```

```
ROCK PAPER SCISSORS GAME
Choose an Option :
1. Rock
2. Paper
3. Sissors
Enter your Choice : 2
You Choose Paper
Computer Choose Scissors
You Lose!
```

```
ROCK PAPER SCISSORS GAME
Choose an Option :
1. Rock
2. Paper
3. Sissors
Enter your Choice : 1
You Choose Rock
Computer Choose Rock
Its a Tie!
```

Sample Program 10 : Banking Program

Code :

```c
void checkBalance(float Balance);
float deposit(float Balance);
float withdraw(float Balance);
int main(){
    int choice = 0;
    float balance = 0.0f;
```

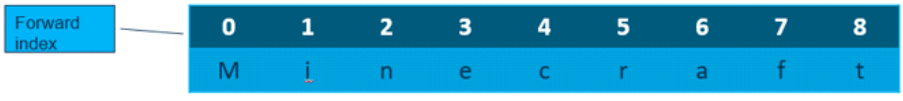
```c
void checkBalance(float Balance);
float deposit(float Balance);
float withdraw(float Balance);
int main(){
    int choice = 0;
    float balance = 0.0f;

    printf("WELCOME TO THE BANK");
    do{
        printf("\nSelect an Option \n");
        printf("1. Check Balance \n");
        printf("2. Deposit Money \n");
        printf("3. Withdraw Money \n");
        printf("4. Exit \n");
        printf("Enter Your Choice : ");
        scanf("%d", &choice);

        switch(choice){
            case 1:
                checkBalance(balance);
                break;
            case 2:
                balance = deposit(balance);
                break;
            case 3:
                balance = withdraw(balance);
                break;
            case 4:
                printf("Thank You!");
                break;
            default:
                printf("Enter a Valid Option.");
                break;
        }
    }while(choice !=4);

    return 0;
}
void checkBalance(float Balance){
    printf("Your Balance is : %.2f \n", Balance);
}
float deposit(float Balance){
    printf("Current Balance : %.2f \n", Balance);
    float deposit = 0.0f;
    printf("Enter Amount to Deposit : ");
    scanf("%f", &deposit);
    Balance += deposit;
    printf("%.2f Deposited Successfully. \n", deposit);
    printf("Your Balance is : %.2f \n", Balance);
    return Balance;
}
float withdraw(float Balance){
    printf("Current Balance : %.2f \n", Balance);
    float withdraw = 0.0f;
    printf("Enter Amount to Withdraw : ");
    scanf("%f", &withdraw);
    if(Balance > withdraw){
        Balance -= withdraw;
        printf("%.2f Withdrawn Successfully. \n", withdraw);
        printf("Your Balance is : %.2f \n", Balance);
    }
    else{
        printf("Not Enough Balance. \n");
    }
    return Balance;
}
```

Output :

```
WELCOME TO THE BANK
Select an Option
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Your Choice : 1
Your Balance is : 0.00

Select an Option
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Your Choice : 2
Current Balance : 0.00
Enter Amount to Deposit : 1000
1000.00 Deposited Successfully.
Your Balance is : 1000.00

Select an Option
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Your Choice : 2
Current Balance : 1000.00
Enter Amount to Deposit : 1000
1000.00 Deposited Successfully.
Your Balance is : 2000.00

Select an Option
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Your Choice : 1
Your Balance is : 2000.00

Select an Option
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Your Choice : 3
Current Balance : 2000.00
Enter Amount to Withdraw : 500
500.00 Withdrawn Successfully.
Your Balance is : 1500.00

Select an Option
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Your Choice : 1
Your Balance is : 1500.00

Select an Option
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Your Choice : 3
Current Balance : 1500.00
Enter Amount to Withdraw : 50000
Not Enough Balance.

Select an Option
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Your Choice : 1
Your Balance is : 1500.00

Select an Option
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Your Choice : 4
Thank You!
```

Indexing,

Indexing starts at 0 and ends at (length - 1).



Arrays,

Array[ ] = { }

Example 1 (Using int),

It is a fixed size collection of elements of the same data type (Similar to a variable, but it holds more than 1 value).

Variable name contains [ ] which specifies that it is an array, and the values are entered in-between { }.

Eg. `int numbers[] = {10, 20, 30, 40, 50};`

Each value in an array is an element.

When an array is passed into a function (like printf()) it decays into a pointer (runs into unexpected behavior).

Print the elements in an array.

Each element is called using its index values (Indexing starts from 0 and ends (len - 1)).

Eg.

```
printf("%d \n", numbers[0]);
printf("%d \n", numbers[1]);
printf("%d \n", numbers[2]);
printf("%d \n", numbers[3]);
printf("%d \n", numbers[4]);
printf("%d \n", numbers[5]);  // Out of index values gives some garbage value (out-of-bounds).
```

```
int main(){
    int numbers[] = {10, 20, 30, 40, 50};
    printf("%d", numbers);          // It gives out a really long funky number.
    // When an array is passed into a function (like printf() ) it decays into a pointer (runs into unexpected behaviour).
```

```c
int main(){
    int numbers[] = {10, 20, 30, 40, 50};
    printf("%d", numbers);          // It gives out a really long funky number.
    // When an array is passed into a function (like printf() ) it decays into a pointer (runs into unexpected behaviour).
    // Each element is called using its index values (Indexing starts from 0 and ends (len - 1) ).

    numbers[4] = 100;               // Changing the value

    printf("%d \n", numbers[0]);
    printf("%d \n", numbers[1]);
    printf("%d \n", numbers[2]);
    printf("%d \n", numbers[3]);
    printf("%d \n", numbers[4]);
    printf("%d \n", numbers[5]);    // Out of index values gives some garbage value (out-of-bounds).

    return 0;
}
```

Example 2 (Using char),

```c
int main(){
    char grades[] = {'A', 'B', 'C', 'D', 'E'};

    grades[4] = 'F';        // Changing the value

    printf("%C \n", grades[0]);
    printf("%C \n", grades[1]);
    printf("%C \n", grades[2]);
    printf("%C \n", grades[3]);
    printf("%C \n", grades[4]);

    return 0;
}
```

Example 3 (Using characters),

```c
int main(){
    // An array of characters is like strings in C
    char name[] = "Person Name";

    name[10] = 'f';         // Changing the value

    printf("%c \n", name[0]);
    printf("%c \n", name[1]);
    printf("%c \n", name[2]);
    printf("%c \n", name[3]);
    printf("%c \n", name[4]);
    printf("%c \n", name[5]);
    printf("%c \n", name[6]);
    printf("%c \n", name[7]);
    printf("%c \n", name[8]);
    printf("%c \n", name[9]);
    printf("%c \n", name[10]);

    return 0;
}
```

Important Points,

- You cannot print an array directly as it will give some long funky number. You have to print it by Index (Calling each element).
- You can change a value in an array by calling its index and assigning the new value to it. Eg : `numbers[4] = 100` (This changes the value at index 4 of the array to 100).

Arrays and User Input,

To take in user data, use scanf function and call the array with its index position.
Or you can take its input and assign it to the array.

```c
int main(){
    int scores[5] = {0};
    int score = 0;
    for(int i = 0; i < 5; i++){
        printf("Enter Score : ");
        scanf("%d", &scores[i]);
    }
    for(int i = 0; i < 5; i++){
        printf("Score : %d \n", scores[i]);
    }

    return 0;
}
```

```c
int main(){
    int scores[5] = {0};
    int score = 0;
    for(int i = 0; i < 5; i++){
        printf("Enter Score : ");
        scanf("%d", &score);
        scores[i] = score;
    }
    for(int i = 0; i < 5; i++){
        printf("Score : %d \n", scores[i]);
    }

    return 0;
}
```

```
    return 0;
}
```

```
    return 0;
}
```

2D Arrays,

An array where each element is an array.

Array[ ][ ] = { { }, { }, { } }

For 2D Arrays, we have to declare the number of columns (the second [ ]).

Useful for Grid, or matrix of data.

Example 1 (Using int),

```
int main(){
    int numbers[][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    for(int i=0; i < 3; i++){
        for(int j=0; j < 3; j++){
            printf("%d ", numbers[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Example 2 (Using char),

```
int main(){
    char numpad[][3] = {{'1', '2', '3'}, {'4', '5', '6'},
                        {'7', '8', '9'}, {'*', '0', '#'}};
    for(int i=0; i < 4; i++){
        for(int j=0; j < 3; j++){
            printf("%c ", numpad[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Arrays with Strings,

Example 1 (Printing array of strings),

```
int main(){
    char fruits[][10] = {"Apple", "Banana", "Coconut"};
    int size = sizeof(fruits) / sizeof(fruits[0]);
    for(int i=0; i < size; i++){
        printf("%s \n", fruits[i]);
    }

    return 0;
}
```

Example 2 (Taking user input),

```
int main(){
    char names[3][25] = {0};
    int size = sizeof(names) / sizeof(names[0]);
    for(int i = 0; i < size; i++){
        printf("Enter a name : ");
        fgets(names[i], sizeof(names[i]), stdin);
        names[i][strlen(names[i]) - 1] = '\0';
    }
    for(int j=0; j < size; j++){
        printf("%s ", names[j]);
    }

    return 0;
}
```

Array of strings behaves similarly like an array of char

```
char fruit[][10] = {
        {'A', 'p', 'p', 'l', 'e', '\0', '\0', '\0', '\0', '\0'},
        {'B', 'a', 'n', 'a', 'n', 'a', '\0', '\0', '\0', '\0'},
        {'C', 'o', 'c', 'o', 'n', 'u', 't', '\0', '\0', '\0'}
};
```

Sample Program 11 : Quiz Game

Code 1 :

```
int main(){
    char quiz1[4][15] = {"A. Jupiter", "B. Saturn", "C. Uranus", "D. Neptune"};
    char quiz2[4][15] = {"A. Mercury", "B. Venus", "C. Earth", "D. Mars"};
    char quiz3[4][15] = {"A. Earth", "B. Mars", "C. Jupiter", "D. Saturn"};
    char quiz4[4][15] = {"A. Yes", "B. No", "C. Maybe", "D. Sometimes"};
    int score = 0;
    char choice = '\0';

    printf("QUIZ GAME \n\n");
    for(int i = 1; i<=4; i++){
        if(i == 1){
            printf("What is the largest planet in the solar system? \n");
            for(int j = 0; j < 4; j++){
```

```
        if(i == 1){
            printf("What is the largest planet in the solar system? \n");
            for(int j = 0; j < 4; j++){
               printf("%s \n", quiz1[j]);
            }
            printf("\nEnter Your Choice : ");
            scanf(" %c", &choice);
            choice=tolower(choice);
            if(choice == 'a'){
                printf("Correct! \n\n");
                score+=1;
            }
            else{
                printf("Wrong \n\n");
            }
        }
        if(i == 2){
            printf("Which is the hottest planet? \n");
            for(int j = 0; j < 4; j++){
               printf("%s \n", quiz2[j]);
            }
            printf("\nEnter Your Choice : ");
            scanf(" %c", &choice);
            choice=tolower(choice);
            if(choice == 'b'){
                printf("Correct! \n\n");
                score+=1;
            }
            else{
                printf("Wrong \n\n");
            }
        }
        if(i == 3){
            printf("What planet has the most moons? \n");
            for(int j = 0; j < 4; j++){
               printf("%s \n", quiz3[j]);
            }
            printf("\nEnter Your Choice : ");
            scanf(" %c", &choice);
            choice=tolower(choice);
            if(choice == 'd'){
                printf("Correct! \n\n");
                score+=1;
            }
            else{
                printf("Wrong \n\n");
            }
        }
        if(i == 4){
            printf("Is the Earth flat? \n");
            for(int j = 0; j < 4; j++){
               printf("%s \n", quiz4[j]);
            }
            printf("\nEnter Your Choice : ");
            scanf(" %c", &choice);
            choice=tolower(choice);
            if(choice == 'b'){
                printf("Correct! \n\n");
                score+=1;
            }
            else{
                printf("Wrong \n\n");
            }
        }
```

```
                printf("Wrong \n\n");
            }
        }
    }
    printf("Your Score is %d out of 4 points.", score);

    return 0;
}
```

Output :

```
QUIZ GAME

What is the largest planet in the solar system?
A. Jupiter
B. Saturn
C. Uranus
D. Neptune

Enter Your Choice : a
Correct!

Which is the hottest planet?
A. Mercury
B. Venus
C. Earth
D. Mars

Enter Your Choice : b
Correct!

What planet has the most moons?
A. Earth
B. Mars
C. Jupiter
D. Saturn

Enter Your Choice : d
Correct!

Is the Earth flat?
A. Yes
B. No
C. Maybe
D. Sometimes

Enter Your Choice : b
Correct!

Your Score is 4 out of 4 points.
```

String functions,

To use these we have to include the <string.h> header file

Usage	Function
To convert an input into upper case	toupper()
To convert an input into lower case	tolower()

Ternary Operator (?),

Shorthand for if-else statements.

(Condition) ? value_if_true : value_if_false;

Example 1 (Using int),

```
int main(){
    int x = 6;
    int y = 7;
    int max = (x > y) ? x : y;
    printf("%d", max);
```

Example 2 (Using bool),

```
int main(){
    bool isOnline = false;

    printf("%s", (isOnline) ? "Online" : "Offline");

    return 0;
```

```c
    int max = (x > y) ? x : y;
    printf("%d", max);

    return 0;
}
```

```c
    printf("%s", (isOnline) ? "Online" : "Offline");

    return 0;
}
```

Example 3 (Taking user input and printing a string),

```c
int main(){
    int age = 0;
    printf("Enter Your Age : ");
    scanf("%d", &age);

    printf("%s", (age < 18) ? "You are a Child." : "You are an Adult.");

    return 0;
}
```

Example 4 (Using Pointers),

```c
int main(){
    int hours = 0;
    int minutes = 0;
    printf("Enter Hour : ");
    scanf("%d", &hours);
    printf("Enter Minute : ");
    scanf("%d", &minutes);
    char *meri = (hours < 12) ? "AM" : "PM";    // Pointer
    printf("%02d:%02d %s", hours, minutes, meri);

    return 0;
}
```

Pointers,

Data type of a pointer is 'char'.
Just put the ternary operator for it.

Typedef,

Reserved keyword that gives an existing datatype a "nickname".
Helps simplify complex types and improves code readability.
typedef existing_type new_name;

Example 1 (Using int),

```c
typedef int number;
int main(){
    number x = 3;
    number y = 4;
    number z = x + y;
    printf("%d", z);

    return 0;
}
```

Example 2 (Using strings),

```c
typedef char String[50];
int main(){
    String name = "My Name";
    printf("%s", name);

    return 0;
}
```

We specified the size `String[50]` so that we don't have to specify it while assigning the variable.
If we use a pointer we don't have to specify the size, typedef char* String; (Advanced topic).

```
}
```

If we use a pointer we don't have to specify the size, typedef char* String; (Advanced topic).

Enum,

A user defined data type that consists of a set of named integer constants.
Benefit : Replaces numbers with readable names.
If value isn't specified, the first constant is 0 by default and the next ones are incremented by 1.
Using Enum, programmers can understand it better.
Enum constants should be in capital letters.
enum tag_name{};

Example 1 (Without specifying enum values),

```c
enum Day{
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
};
int main(){
    enum Day today = SUNDAY; // Sunday Has a value of 0
    printf("%d \n", today);
    enum Day today1 = MONDAY; // Monday Has a value of 1
    printf("%d \n", today1);
    enum Day today2 = TUESDAY; // Tuesday Has a value of 2
    printf("%d \n", today2);
    enum Day today3 = WEDNESDAY; // Wednesday Has a value of 3
    printf("%d \n", today3);
    enum Day today4 = THURSDAY; // Thursday Has a value of 4
    printf("%d \n", today4);
    enum Day today5 = FRIDAY; // Friday Has a value of 5
    printf("%d \n", today5);
    enum Day today6 = SATURDAY; // Saturday Has a value of 6
    printf("%d \n", today6);

    return 0;
}
```

Example 2 (enum values specified),

```c
enum Day{
    SUNDAY = 1, MONDAY = 2, TUESDAY = 3, WEDNESDAY = 4,
    THURSDAY = 5, FRIDAY = 6, SATURDAY = 7
};
int main(){
    enum Day today = SUNDAY; // Sunday Has a value of 1
    printf("%d \n", today);
    enum Day today1 = MONDAY; // Monday Has a value of 2
    printf("%d \n", today1);
    enum Day today2 = TUESDAY; // Tuesday Has a value of 3
    printf("%d \n", today2);
    enum Day today3 = WEDNESDAY; // Wednesday Has a value of 4
    printf("%d \n", today3);
    enum Day today4 = THURSDAY; // Thursday Has a value of 5
    printf("%d \n", today4);
    enum Day today5 = FRIDAY; // Friday Has a value of 6
    printf("%d \n", today5);
    enum Day today6 = SATURDAY; // Saturday Has a value of 7
    printf("%d \n", today6);

    return 0;
}
```

Example 3 (typedef + enum),

Example 3 (typedef + enum),

```c
typedef enum {
    SUNDAY = 1, MONDAY = 2, TUESDAY = 3, WEDNESDAY = 4,
    THURSDAY = 5, FRIDAY = 6, SATURDAY = 7
}Day;
int main(){
    Day today = SUNDAY; // Sunday Has a value of 1
    printf("%d \n", today);
    Day today1 = MONDAY; // Monday Has a value of 2
    printf("%d \n", today1);
    Day today2 = TUESDAY; // Tuesday Has a value of 3
    printf("%d \n", today2);
    Day today3 = WEDNESDAY; // Wednesday Has a value of 4
    printf("%d \n", today3);
    Day today4 = THURSDAY; // Thursday Has a value of 5
    printf("%d \n", today4);
    Day today5 = FRIDAY; // Friday Has a value of 6
    printf("%d \n", today5);
    Day today6 = SATURDAY; // Saturday Has a value of 7
    printf("%d \n", today6);

    return 0;
}
```

If you use typedef, you dont have to specify enum.

Example 4,

```c
typedef enum {
    SUNDAY = 1, MONDAY = 2, TUESDAY = 3, WEDNESDAY = 4,
    THURSDAY = 5, FRIDAY = 6, SATURDAY = 7
}Day;
int main(){
    Day today = MONDAY;

    if(today == SUNDAY || today == SATURDAY){
        printf("It is a Weekend.");
    }
    else{
        printf("It is a Weekday.");
    }

    return 0;
}
```

Example 5 (Using Switch and Function),

```c
typedef enum{
    SUCCESS, FAILURE, PENDING
}Status;
void connectStatus(Status status){
    switch(status){
        case SUCCESS:
            printf("Connection was Successful.");
            break;
        case FAILURE:
            printf("Could not connect.");
            break;
        case PENDING:
            printf("Connecting...");
            break;
    }
```

```
            printf("Connecting...");
            break;
    }
};
int main(){
    Status status = PENDING;
    connectStatus(status);

    return 0;
}
```

Structs,

A custom container that holds multiple pieces of related information.
Similar to Objects in other languages.
It is like a blueprint.
To access an element in the struct, use '.'
If we just create a struct and not assign it any values and if we call it gives undefined behavior (if we just do `tag_name variable_name;`).
To create an empty struct just do `tag_name variable_name = {0};` so that it resets all the va-lue to 0.

struct tag_name{};

Example 1 (Creating and Using Structs),

```
typedef struct{
    char name[50];
    int age;
    float gpa;
    bool isFulltime;
}Student;
void printstudent(Student student){
    // To access an element in the struct, use '.'
    printf("Name : %s \n", student.name);
    printf("Age : %d \n", student.age);
    printf("GPA : %.2f \n", student.gpa);
    printf("Full Time : %s \n", (student.isFulltime) ? "Yes" : "No");
    printf("\n");
};
int main(){
    Student student1 = {"Spongebob", 30, 2.5, true};
    Student student2 = {"Patrick", 32, 1.0, false};
    Student student3 = {"Squidward", 48, 3.2, false};
    Student student4 = {0};
    printstudent(student1);
    printstudent(student2);
    printstudent(student3);

    return 0;
}
```

Example 2 (Assigning values to an empty struct),

```
typedef struct{
    char name[50];
    int age;
    float gpa;
    bool isFulltime;
}Student;
int main(){
    Student student = {0};
    strcpy(student.name, "Sandy");
    student.age = 27;
    student.gpa = 4.0;
```

```c
    strcpy(student.name, "Sandy");
    student.age = 27;
    student.gpa = 4.0;
    student.isFulltime = true;
    printf("Name : %s \n", student.name);
    printf("Age : %d \n", student.age);
    printf("GPA : %.2f \n", student.gpa);
    printf("Full Time : %s \n", (student.isFulltime) ? "Yes" : "No");

    return 0;
}
```

Example 3 (Taking user input and assigning into struct),

```c
typedef struct{
    char name[50];
    int age;
    float gpa;
    bool isFulltime;
}Student;
int main(){
    Student student = {0};
    char name[50] = "";
    char fulltime = '\0';
    printf("Enter Student Name : ");
    fgets(name, sizeof(name), stdin);
    name[strlen(name) - 1] = '\0';
    strcpy(student.name, name);
    printf("Enter Student Age : ");
    scanf("%d", &student.age);
    printf("Enter Student GPA : ");
    scanf("%f", &student.gpa);
    printf("Is the Student Full Time? (Y/N) : ");
    scanf(" %c", &fulltime);
    if(fulltime == 'Y' || fulltime == 'y'){student.isFulltime = true;}
    else{student.isFulltime = false;}
    printf("Name : %s \n", student.name);
    printf("Age : %d \n", student.age);
    printf("GPA : %.2f \n", student.gpa);
    printf("Full Time : %s \n", (student.isFulltime) ? "Yes" : "No");

    return 0;
}
```

Array of structs,

Array where each element contains a struct {}.

Helps organize and groups together related data.

```c
typedef struct{
    char model[25];
    int year;
    int price;
}Car;
int main(){
    Car cars[] = {{"Mustang", 2025, 32000},
    {"Corvette", 2026, 68000},
    {"Challenger", 2024, 28000}};
    int number = sizeof(cars) / sizeof(cars[0]);

    for(int i = 0; i < number; i++){
        printf("Model : %s  Year : %d   Price : $%d \n", cars[i].model, cars[i].year, cars[i].price);
    }
    return 0;
```

```
    for(int i = 0; i < number; i++){
        printf("Model : %s  Year : %d   Price : $%d \n", cars[i].model, cars[i].year, cars[i].price);
    }
    return 0;
}
```

Pointers,

A variable that stores the memory address of another variable.
Benefit : They help avoid wasting memory by allowing you to pass the address of a large data structure instead of copying the entire data.
`%p` is used to print the pointer address.
`&` gives the address (Addresses are hexadecimal values).
`*` is known as the dereference operator, and then you put a 'p' to specify that it is a pointer.
You can put the `*` either a suffix after the data type or as a prefix before the variable name.

```
int main(){
    int age = 25;
    // `%p` is used to print the pointer address.
    // `&` gives the address  (Addresses are hexadecimal values).
    int *pAge = &age;
    // `*` is known as the dereference operator, and then you put a 'p' to specify that it is a pointer.
    // You can put the `*` either a suffix after the data type or as a prefix before the variable name.
    printf("%p \n", &age);
    printf("%p", pAge);

    return 0;
}
```

Example 1,

Usually functions are pass by value.
We have to do Pass by reference.
`(*age)` we dereference it and then increment it by 1 or else it will increment the address by 1.
So we dereference it and get the value and then increment it by 1
'*age' is within () is because of operator precedence (without () it will increment the address first and then dereference it.

```
void birthday(int* age){
    (*age)++;
};
int main(){
    int age = 25;
    int *pAge = &age;

    birthday(pAge);
    printf("You are %d years old", age);

    return 0;
}
```

We don't have to necessarily create a variable for the pointer and then pass it into a function, you can directly pass it in using `&variable_name`

```
void birthday(int* age){
    (*age)++;
};
int main(){
    int age = 25;

    birthday(&age);
    printf("You are %d years old", age);

    return 0;
}
```

Write Files,

To work with Files, there is a built in struct, a file, provided by the standard input output header file.
Data type - 'FILE' (All caps).
We will create a pointer to the file struct.
fopen() - File Open (We add the file path followed by the mode in this function) (Always close the file, use `fclose(pointer);`).
If unable to create or open a file it returns a value of NULL.
Mode,

Mode	Explanation	Example
r	Read mode	.

Mode,

Mode	Explanation	Example
r	Read mode	.
w	Write mode	.

To write into a text file use `fprint(file_pointer, "%format_specifer", variable_name);` where the 'f' infront of 'printf' stands for file.
Absolute File Path - The entire location to the file. Eg "C:\\Users\\user\\Folder\\File.txt" (Use double \ cause of escape sequence),
Used if you want to store the file in a different folder and not in the folder the program file is in.
Relative File Path - Just the file name and the file should be in the same folder as the program file (It will raise an error if file is not found in read mode).

```c
int main(){
    FILE *pFile = fopen("output.txt", "w");
    char text[] = "Idk what do I type here \nso you tell me.";
    if(pFile == NULL){
        printf("Error opening file.");
        return 1;
    }
    fprintf(pFile, "%s", text);
    printf("File Was Written Successfully");
    fclose(pFile);

    return 0;
}
```

Read Files,
Buffer is used to temporarily store data.
We have to specify the size of the buffer (in bytes) (best is 1024 bytes = 1Kb).
We use a while loop to read the data from the file.
Use fgets function so once it finishes reading the full file it will return a NULL value (while loops runs if its not NULL).
`fgets(buffer, sizeof(buffer), pFile`

```c
int main(){
    FILE *pFile = fopen("input.txt", "r");
    char buffer[1024] = {0};
    if(pFile == NULL){
        printf("Error Opening File.");
        return 1;
    }
    while(fgets(buffer, sizeof(buffer), pFile) != NULL){
        printf("%s", buffer);
    }
    fclose(pFile);

    return 0;
}
```

For malloc, calloc and realloc functions we have to include standard library header file <stdlib.h>

malloc function,
malloc(bytes);
A function in C that dynamically allocates a specified number of bytes in memory.
You calculate the space using the mallco function and the variable name should be a pointer, mallco(number * sizeof(char))
char - 1 byte
Int - 4 bytes
If the function fails, it returns a value of NULL. And if we try to dereference a NULL Pointer, it will cause a Segmentation Fault (which is dangerous).
Segmentation fault occurs when a program attempts to access a memory location that is not allowed to access, or attempts to access a memory location in a way that is not allowed (for Eg : attempting to write a read-only location, or to overwrite part of the operating system).

If the function fails, it returns a value of NULL. And if we try to dereference a NULL pointer, it will cause a Segmentation Fault (which is dangerous).

Segmentation fault occurs when a program attempts to access a memory location that is not allowed to access, or attempts to access a memory location in a way that is not allowed (for Eg : attempting to write a read-only location, or to overwrite part of the operating system).

The Memory that we reserve is from the location known as 'Heap', in most situations when we use the memory it is from the place called 'Stack'.
Once we are done using the reserved space, we have to return the space back to the os, we can do that using the free function `free(variable_name);`.
We also have to reset the pointer to avoid dangling pointers (dangling pointer are pointers that point to a location that we arent using anymore, we do this by doing `variable_name = NULL;` .

```c
int main(){
    int number = 0;
    printf("Enter the Number of Grades : ");
    scanf("%d", &number);

    char *grades = malloc(number * sizeof(char));
    if(grades == NULL){
        printf("Memory Allocation Failed.");
        return 1;
    }
    for(int i = 0; i < number; i++){
        printf("Enter Grade #%d : ", i+1);
        scanf(" %c", &grades[i]);
    }
    for(int i = 0; i<number; i++){
        printf("Grade #%d : %c \n", i+1, grades[i]);
    }

    free(grades);    // returning the memory space back to the os.
    grades = NULL;   // avoides dangling pointers

    return 0;
}
```

calloc function,

Contiguous Allocation.
Allocates memory dynamically and sets all allocated bytes to 0.
malloc() is faster, but calloc() leads to less bugs.

malloc function if printed without assigning values to it, they return undefined behavior. While in calloc function it sets all allocated bytes to 0 which leads to lesser bugs.

calloc(number_of_elements, size);

```c
int main(){
    int number = 0;
    printf("Enter Number of Players : ");
    scanf("%d", &number);

    int *scores = calloc(number, sizeof(int));
    if(scores == NULL){
        printf("Memory Allocation Failed.");
        return 1;
    }
    for(int i = 0; i<number; i++){
        printf("Enter Score of Player #%d : ", i+1);
        scanf("%d", &scores[i]);
    }
    for(int i = 0; i<number; i++){
        printf("%d ", scores[i]);
    }
```

```
    for(int i = 0; i<number; i++){
        printf("%d ", scores[i]);
    }
    free(scores);   // returning the memory space back to the os.
    scores = NULL;  // avoids dangling pointers

    return 0;
}
```

realloc function,

Reallocation.
Resize previously allocated memory.
realloc will also free old memory.
We can make the memory bigger or smaller.

realloc(pointer, size_in_bytes);

```
int main(){
    int number = 0;
    printf("Enter the Number of Prices : ");
    scanf("%d", &number);

    float *prices = malloc(number * sizeof(float));
    if(prices == NULL){
        printf("Error Allocating Memory");
        return 1;
    }
    for(int i = 0; i < number; i++){
        printf("Enter Price of Item #%d : ", i+1);
        scanf("%f", &prices[i]);
    }

    int newNumber = 0;
    printf("Enter a New Number Of Prices : ");
    scanf("%d", &newNumber);

    float *temp = realloc(prices, newNumber * sizeof(float));     // Temp means temporary
    if(temp == NULL){
        printf("Could Not Reallocate Memory");
    }
    else{
        prices = temp;

        for(int i = number; i < newNumber; i++){
            printf("Enter Price of Item #%d : ", i+1);
            scanf("%f", &prices[i]);
        }
        for(int i = 0; i < newNumber; i++){
            printf("$%.2f ", prices[i]);
        }
    }

    free(prices);
    prices = NULL;

    return 0;
}
```

Sample Program 12 : Digital Clock,

Code:

Code:

```c
int main(){
    time_t rawtime = 0;
    // Using this cause int has a limit on how big an integer is, and this holds Unix Epoch.
    // Unix Epoch is around Jan 1 1970. (It measures how many seconds has passed since this date and that date is used as a reference point).
    struct tm *pTime = NULL;
    bool isRunning = true;

    printf("Digital Clock \n");
    while(isRunning){
        time(&rawtime);
        pTime = localtime(&rawtime);
        printf("\r%02d:%02d:%02d", pTime->tm_hour, pTime->tm_min, pTime->tm_sec);
        // the `->` means "dereference the operator first and then give the current value".
        // `\r` means carriage return which means the cursor returns to the beginning.
        Sleep(1000);
    }

    return 0;
}
```

Extra:

```c
struct tm {
    int tm_sec;   // seconds after the minute [0-60]
    int tm_min;   // minutes after the hour [0-59]
    int tm_hour;  // hours since midnight [0-23]
    int tm_mday;  // day of the month [1-31]
    int tm_mon;   // months since January [0-11]
    int tm_year;  // years since 1900
    int tm_wday;  // days since Sunday [0-6]
    int tm_yday;  // days since January 1 [0-365]
    int tm_isdst; // daylight saving time flag
};
```